

Recursive GCD Timed + Optimization

Emmanuel Cano

Objective

- Analyze compiler-generated assembly for the function:
 - gcd_recurs()
- Compare performance on ARM (Raspberry Pi) and x86-64 (Linux Intel) by using the chrono library to time results.
- Apply manual optimizations to improve runtime efficiency and explain improvements.
- Compare original vs. optimized versions for both systems.

C++ Code For Both Systems: gcd_recurs()

```
#include <iostream>
#include <chrono>

using namespace std;
using namespace std::chrono;

int gcd_recurs(int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd_recurs(b, a % b);
}

int main() {
    int a = 47373888;
    int b = 18883888;

    // Start timing
    auto start = high_resolution_clock::now();

    int result = gcd_recurs(a, b);

    // End timing
    auto end = high_resolution_clock::now();

    // Compute elapsed time and cast to nanoseconds
    auto duration = duration_cast<nanoseconds>(end - start);

    cout << a << "-" << b << "," << result << "," << duration.count() << "\n";

    return 0;
}
```

Figure 1. C++ code of Euclidean algorithm for finding the greatest common divisor implemented recursively.

Methodology

Experimental Setup

Performance measurements were conducted on two platforms: a Raspberry Pi 5 (ARMv8-A) and a Linux Intel system (x86-64). Both systems executed the same C++ implementation of a recursive Euclidean algorithm (`gcd_rekurs`) to ensure functional equivalence across architectures. Timing measurements focused exclusively on the execution of the GCD computation.

Table 1. System Specification

Component	Linux Intel (Specification)	Raspberry Pi (Specification)
Operating System	Ubuntu 25.10 (64-bit)	Debian GNU/Linux 12 (bookworm)
Processor	13th Gen Intel(R) Core(™) i7-1355U	Cortex-A76
Architecture	x86_64	aarch64
Compiler	g++ 15.2.0	g++ 12.2.0
C++ Standard	C++ 17	C++ 17

Test Case Organization

Four input pairs were selected to represent different algorithmic behaviors:

1. $a = 13, b = 17$: Relatively Prime Numbers - Tests that the algorithm correctly reaches $\text{GCD}(k, 1)$ and eventually $\text{GCD}(1, 0)$ when the inputs share no common factors other than 1.
2. $a = 60, b = 20$: Immediate Termination - Test the case where the larger number (a) is a direct multiple of the smaller number (b), resulting in the quick remainder $a \bmod b = 0$ on the first step.
3. $a = 55, b = 34$: Worst Case Performance - Test the maximum number of recursive steps for inputs of this magnitude. 55 and 34 are consecutive Fibonacci numbers(F10 and F9), which are known to be worst-case inputs for the Euclidean Algorithm.
4. $a = 47373888, b = 18883888$: Large General Case - Test the function's correctness and performance under typical, large inputs.

Because the input values a and b were hard-coded into the C++ source, a separate executable was compiled for each test case. This approach ensured that:

- No runtime input parsing affected measurements

- Each executable represented a single, fixed workload
- Compiler output and assembly could be examined independently per test case

Compilation Strategy

All C++ programs were compiled without any optimization flags, including -O0.

This decision was intentional and motivated by the goals of the experiment:

- To observe baseline compiler-generated assembly
- To prevent the compiler from applying automatic optimizations that could obscure:
 - Stack frame behavior
 - Register allocation decisions
 - Recursive call overhead
- To ensure that any observed performance improvements were solely the result of manual assembly-level optimization

Both platforms used the GNU C++ compiler (g++) with the C++17 standard.

Timing Methodology

Execution time was measured inside the C++ program using the `std::chrono` library:

- `std::chrono::high_resolution_clock` was used to record start and end timestamps
- Elapsed time was computed using `std::chrono::duration_cast<std::chrono::nanoseconds>`
- Only the execution of the `gcd_rekurs` function was timed

Timing output was printed to standard output for collection by an external script.

Automation and Repetition

To reduce noise and improve measurement reliability:

- A Python script was used to automate execution
- Each executable (corresponding to one test case) was run 10 times
- The script recorded all timing results and computed the average execution time for each test case

This process was repeated for:

- Original compiler-generated implementations
- Manually optimized implementations

The use of automation ensured consistent execution order and eliminated manual measurement error.

Data Visualization

Performance results were visualized using Matplotlib:

- Bar charts were generated to compare:
 - Original vs optimized implementations
 - ARM vs x86-64 performance
- Charts display average execution time in nanoseconds
- Raw timing data and averages are provided in the Appendix to support reproducibility

Methodological Limitations

This experiment measures the execution time of a single invocation of the GCD function per run. While repeated execution and averaging mitigate some variability, the following factors may still influence results:

- Timing overhead from `std::chrono`
- Cache effects and branch prediction warm-up
- Operating system scheduling noise

Despite these limitations, the methodology is sufficient for relative performance comparison between original and optimized implementations on the same platform, which is the primary goal of this study.

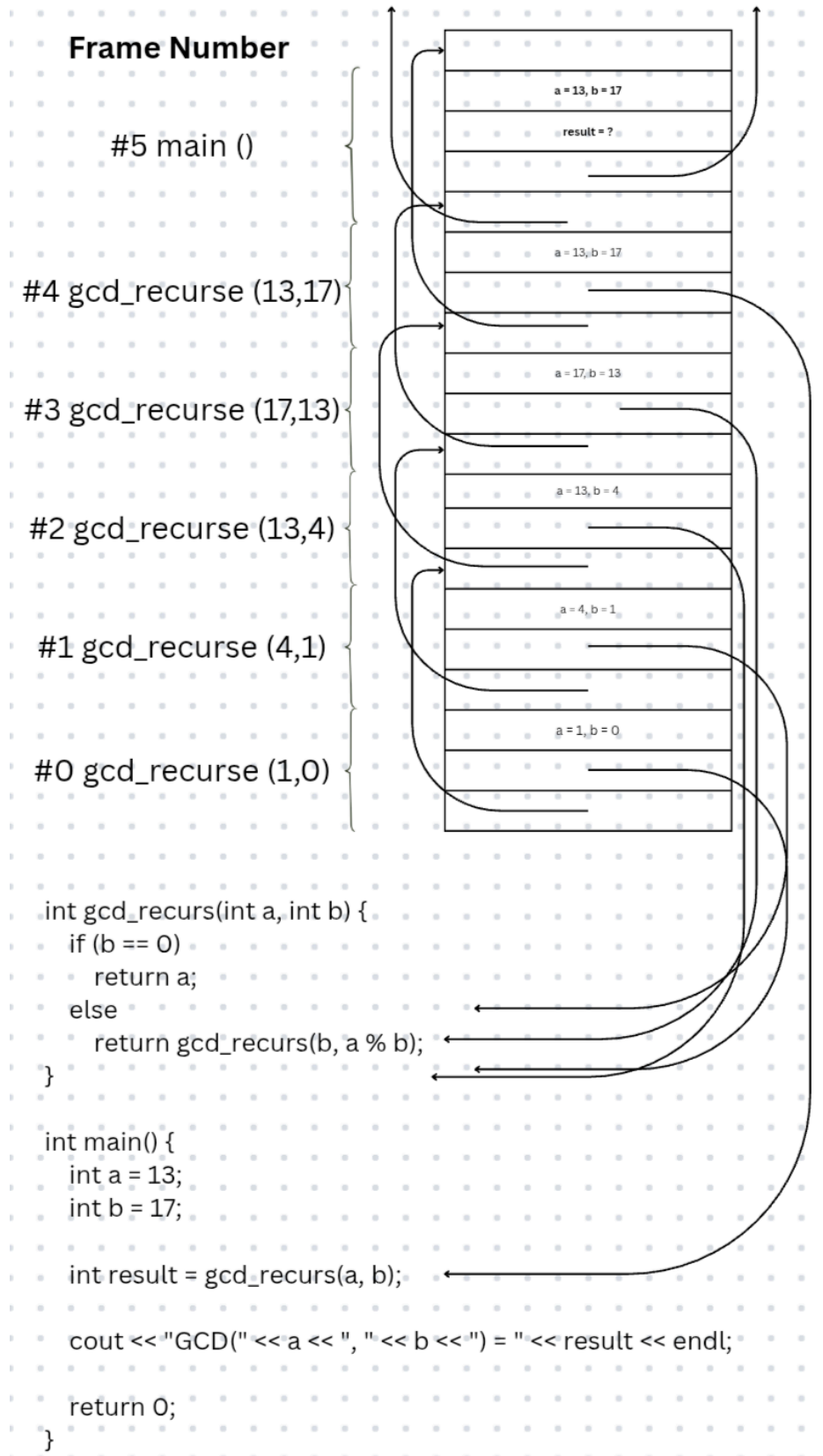


Figure 2. Recursive Call Trace of the Euclidean Algorithm Implemented in `gcd_recurs(a, b)`

Table 2. Execution Time of gcd_recurs on Raspberry Pi (ARM) and Linux Intel (x86-64)

Test Cases (a,b)	Raspberry Pi (ns)	Linux Intel (ns)
a = 13 b = 17	155.4	197.4
a = 60 b = 20	114.9	152.5
a = 55 b = 34	192.6	255
a = 47373888 b = 18883888	233.3	337.9

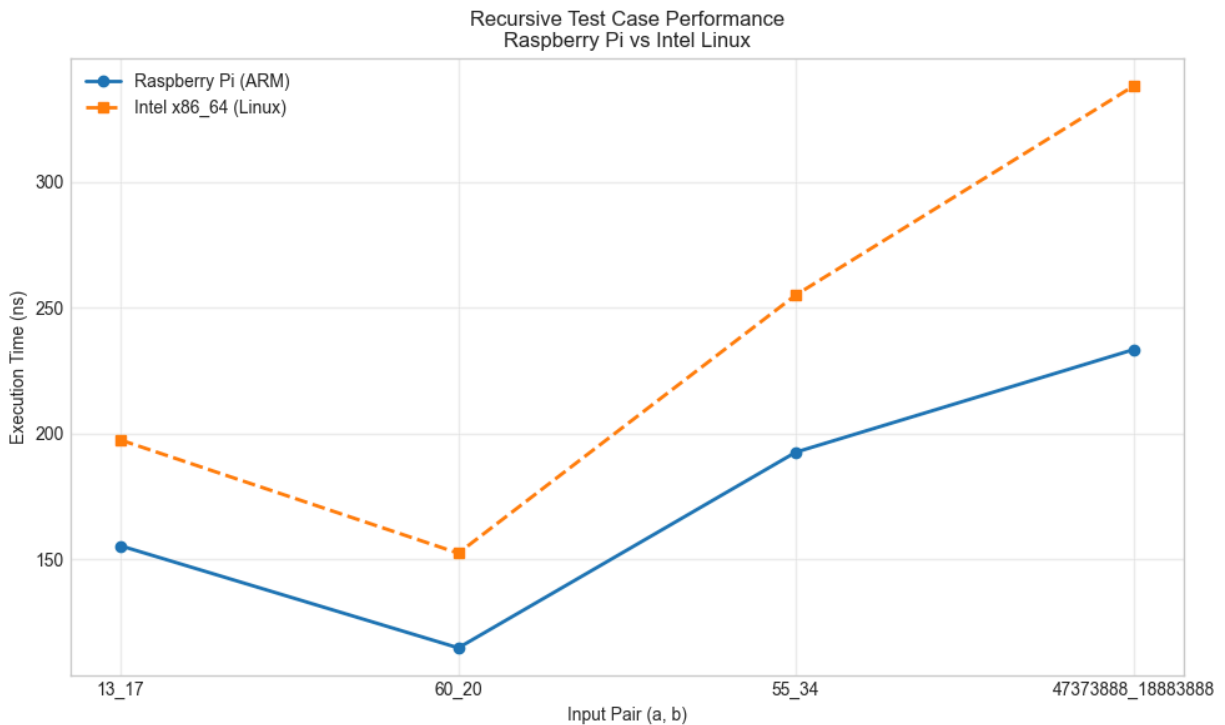


Figure 3. GCD Performance Comparison Between ARM Raspberry Pi and Linux Intel.

Raspberry Pi Performance

Raspberry Pi s File (Snippet): Original gcd_recurs()

```
.text
.align 2
.global _Z10gcd_recursii
.type _Z10gcd_recursii, %function
_Z10gcd_recursii:
```

```

.LFB2046:
.cfi_startproc
stp    x29, x30, [sp, -32]!
.cfi_def_cfa_offset 32
.cfi_offset 29, -32
.cfi_offset 30, -24
mov     x29, sp
str     w0, [sp, 28]
str     w1, [sp, 24]
ldr     w0, [sp, 24]
cmp     w0, 0
bne     .L5
ldr     w0, [sp, 28]
b       .L6
.L5:
ldr     w0, [sp, 28]
ldr     w1, [sp, 24]
sdiv    w2, w0, w1
ldr     w1, [sp, 24]
mul     w1, w2, w1
sub     w0, w0, w1
mov     w1, w0
ldr     w0, [sp, 24]
bl      _Z10gcd_rekursii
nop
.L6:
ldp     x29, x30, [sp], 32
.cfi_restore 30
.cfi_restore 29
.cfi_def_cfa_offset 0
ret
.cfi_endproc
.LFE2046:
.size   _Z10gcd_rekursii, .-_Z10gcd_rekursii

```

Code Snippet Explanation

- `stp x29, x30, [sp, -32]!` : Push frame pointer (x29) and return address (x30)
- `mov x29, sp` : New frame pointer
- `str w0, [sp, 28]` : Store 'a' (first argument) in stack
- `str w1, [sp, 24]` : Store 'b' (second argument) in stack
- `ldr w0, [sp, 24]` : Load b
- `cmp w0, 0` : Compare b with 0
- `bne .L5` : If b != 0, go to .L5
- `ldr w0, [sp, 28]` : Else, load a (the result)
- `b .L6` : Jump to end
- `ldr w0, [sp, 28]` : Load a
- `ldr w1, [sp, 24]` : Load b
- `sdiv w2, w0, w1` : $w2 = a / b$ (integer division)
- `ldr w1, [sp, 24]` : Load b again
- `mul w1, w2, w1` : $w1 = (a / b) * b$
- `sub w0, w0, w1` : $w0 = a - (a / b) * b = a \% b$
- `mov w1, w0` : Put remainder in w1 (b for recursive call)

- `ldr w0, [sp, 24] : Load old b (now the new a)`
- `bl _Z10gcd_recursei: Call gcd_recur(b, a % b)`
- `ldp x29, x30, [sp], 32 : Restore frame and return`

Issues:

- Too many memory loads and stores. It saves and reloads a and b from stack repeatedly, even though they fit in registers.
- Redundant reload of b before mul. We can reuse the value in a register.
- The remainder computation $(a - (a/b)*b)$ can be done more efficiently.

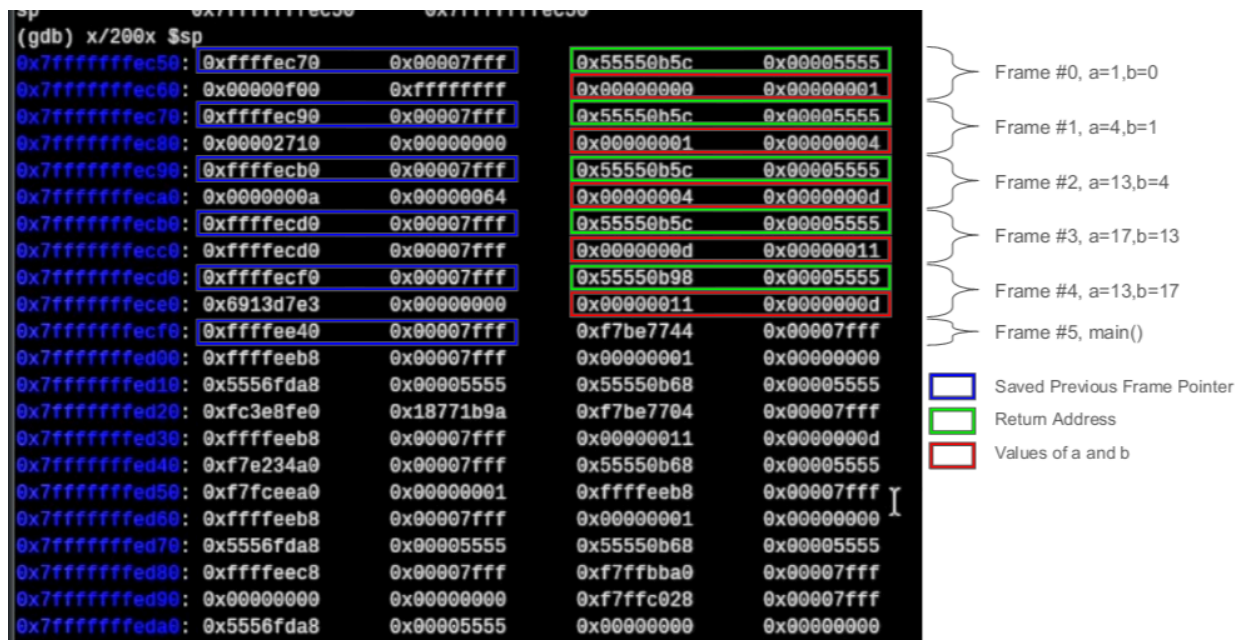


Figure 4. Stack Frame Layout During Recursive Calls to `gcd_recur` (Raspberry Pi, GDB Inspection)

Raspberry Pi s File (Snippet): Optimized `gcd_recur()`

```

_Z10gcd_recursei:
    .cfi_startproc
    stp     x29, x30, [sp, -16]!
    mov     x29, sp

    cmp     w1, 0
    beq     .Ldone

    sdiv    w2, w0, w1
    msub    w3, w2, w1, w0
    mov     w0, w1
    mov     w1, w3
    bl     _Z10gcd_recursei

.Ldone:
    ldp     x29, x30, [sp], 16

```



```

ret

.cfi_endproc
.LFE2046:
.size _Z10gcd_recurcii, .-_Z10gcd_recurcii

```

Explanation of Optimized Code Block

- `stp x29, x30, [sp, -16]! + mov x29, sp` : Saves the frame pointer (x29) and link register (x30) on the stack. Uses only 16 bytes of stack space (minimal possible), improving efficiency.
- `cmp w1, 0` : Compares parameter b (w1) to zero.
- `beq .Ldone` : If `b == 0`, we branch to `.Ldone` to return a (w0) immediately. This is the base case of recursion.
- `sdiv w2, w0, w1` : Performs signed integer division (`w2 = a / b` (quotient)). `sdiv` is a hardware-supported instruction on ARMv8-A, significantly faster than software division, though still multi-cycle.
- `msub w3, w2, w1, w0` : Computes the remainder (`a % b`) efficiently (`w3 = w0 - (w2 * w1)`). This is a fused multiply/subtract instruction which does multiplication and subtraction in one cycle. Using a separate destination register (w3).
- `mov w0, w1 + mov w1, w3` : Prepares recursive call arguments: `a = b` and `b = remainder`. This matches `gcd_recursive(b, a % b)` in C++.
- `bl _Z10gcd_recurcii` : Recursive call. `bl` stores the return address in x30. Since we saved it in the prologue, recursion can safely unwind later.
- `.Ldone: + ldp x29, x30, [sp], 16 + ret` : Restores frame pointer and link register, then returns to the caller. The correct result is already in w0, since the recursive call (or base case) leaves it there.

Results:

- Faster execution since it stays entirely in registers (no stack use except for return addresses).
- Less instruction overhead per recursive call because it fuses multiplication and subtraction into one instruction,
- Minimizes stack frame setup to the smallest possible safe amount.

Table 3. Raspberry Pi (ARM) Recursive GCD: Original vs Manually Optimized Assembly

Test Cases (a,b)	Raspberry Pi (ns)	Optimized-Raspberry Pi (ns)
a = 13 b = 17	155.4	140.7
a = 60	114.9	103.6

b = 20		
a = 55 b = 34	192.6	137.1
a = 47373888 b = 18883888	233.3	164.9

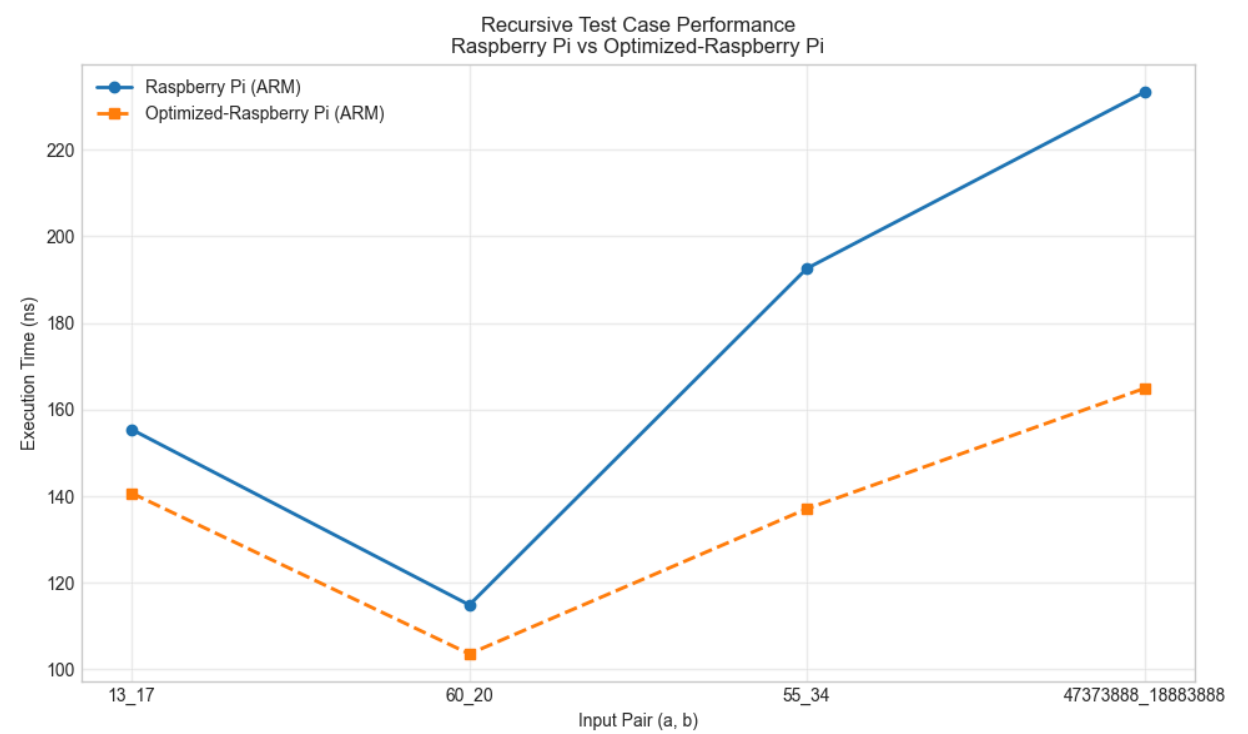


Figure 5. Average Execution Time of Recursive GCD on Raspberry Pi (ARM), Original vs Optimized Assembly

Linux Intel Performance

Linux Intel s File (Snippet): Original gcd_recurs()

```
.text
.globl _Z10gcd_recursii
.type _Z10gcd_recursii, @function
_Z10gcd_recursii:
.LFB2308:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
```

```

    subq    $16, %rsp
    movl    %edi, -4(%rbp)
    movl    %esi, -8(%rbp)
    cmpl    $0, -8(%rbp)
    jne     .L5
    movl    -4(%rbp), %eax
    jmp     .L6
.L5:
    movl    -4(%rbp), %eax
    cltd
    idivl   -8(%rbp)
    movl    -8(%rbp), %eax
    movl    %edx, %esi
    movl    %eax, %edi
    call    _Z10gcd_recurzii
    nop
.L6:
    leave
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
.LFE2308:
    .size   _Z10gcd_recurzii, .-_Z10gcd_recurzii

```

Code Snippet Explanation

- `pushq %rbp + movq %rsp, %rbp + subq $16, %rsp` : Standard function prologue.
- `movl %edi, -4(%rbp)` : Argument a is passed to register edi = first integer (a)
- `movl %esi, -8(%rbp)` : Argument b is passed to register esi = second integer (b).
These are stored on the stack at offsets -4 and -8.
- `cmpl $0, -8(%rbp)` : Compares b to 0
- `jne .L5` : If b != 0, jumps to .L5 (recursive case)
- `movl -4(%rbp), %eax` : Load a into %eax
- `cltd` : Sign-extend eax into edx:eax before division
- `idivl -8(%rbp)` : Divide edx:eax by b; quotient in eax, remainder in edx
- `movl -8(%rbp), %eax` : Reload b into eax
- `movl %edx, %esi` : Put remainder (a % b) into 2nd argument (b)
- `movl %eax, %edi` : Put old b into 1st argument (a)
- `call gcd_recur` : Recursive call

Issues:

- Stack frame setup uses push / pop each call which adds overhead.
- Unnecessary stores such as saving a and b to memory causing slow recursion.
- `cdq + idiv` does division and modulus separately resulting in extra micro-ops.

Linux Intel: gcd_recur In Memory

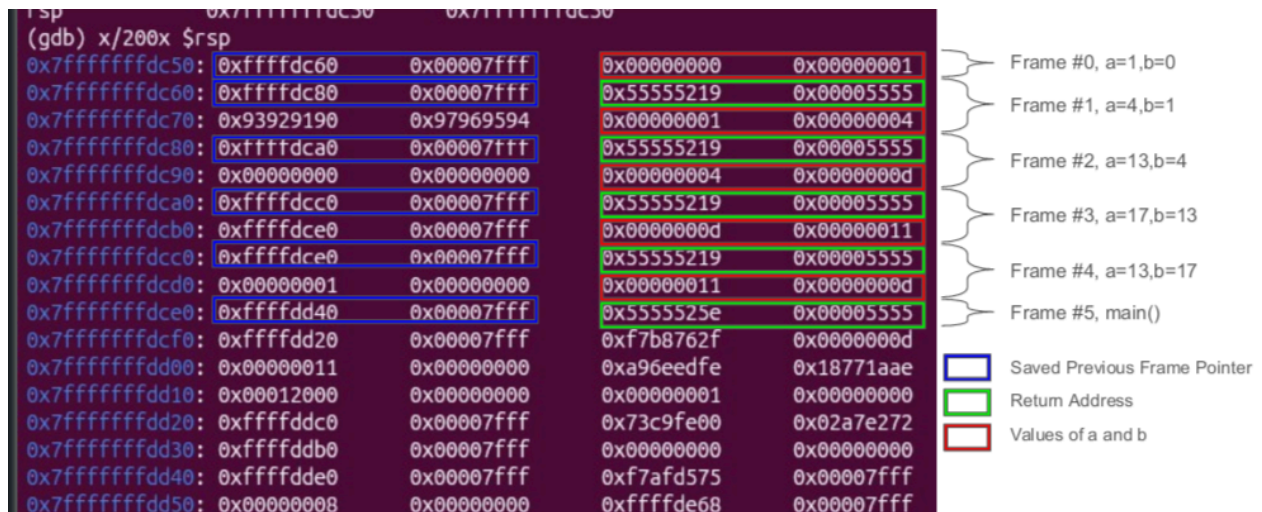


Figure 6. Stack Frame Layout During Recursive Calls to gcd_recurs (Raspberry Pi, GDB Inspection)

Linux Intel s File (Snippet): Optimized gcd_recurs()

Since optimization was already made to Raspberry Pi to improve recursive function, optimization for Linux Intel will be improving from recursion to iteration to show how much faster iteration is compared to recursion. Linux Intel was also chosen for this because it had worse results than Raspberry Pi when comparing original recursion function time results.

```

_Z10gcd_recursii:
    .cfi_startproc
.Loop:
    testl    %esi, %esi
    je       .Return
    movl     %edi, %eax
    xorl     %edx, %edx
    divl     %esi
    movl     %esi, %edi
    movl     %edx, %esi
    jmp      .Loop
.Return:
    movl     %edi, %eax
    ret
    .cfi_endproc
.LFE2308:
    .size    _Z10gcd_recursii, .-_Z10gcd_recursii

```

Explanation of Optimized Code Block

- `testl %esi, %esi` : check if b == 0. a = edi, b = esi
- `je .Return` : if b == 0, return a. This is the base case.
- `movl %edi, %eax` : `eax = a`
- `xorl %edx, %edx` : clear edx for unsigned div

- `divl %esi : divide eax by b and store remainder in edx`
- `movl %esi, %edi : a = b`
- `movl %edx, %esi : b = a % b`
- `jmp .Loop : Loops back to the top instead of calling gcd_rekurs.`
- `movl %edi, %eax : return a`

Results:

- No stack frame setup (`pushq %rbp, movq %rsp,%rbp`, etc.)
- No recursive calls (tail recursion manually flattened into a loop)
- Uses registers only (no memory access)
- Since all test inputs are positive integers, unsigned division (`div`) produces identical results to signed division (`idiv`) while avoiding sign-extension overhead.
- Only 7 total instructions inside the loop

Table 4. Linux Intel (x86-64) GCD: Recursive Implementation vs Iterative Assembly Optimization

Test Cases (a,b)	Linux Intel (ns)	Optimized-Linux Intel (ns)
a = 13 b = 17	197.4	170.2
a = 60 b = 20	152.5	160
a = 55 b = 34	255	186.6
a = 47373888 b = 18883888	337.9	211.3

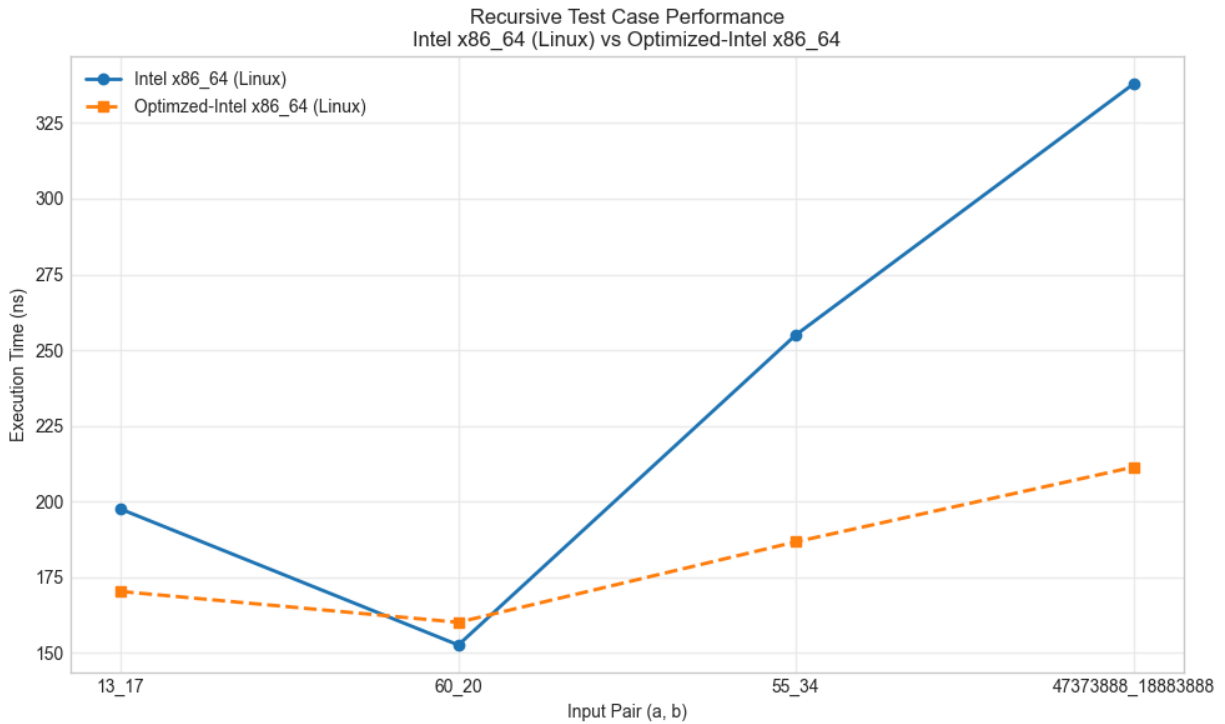


Figure 7. Average Execution Time of GCD on Linux Intel (x86-64), Recursive vs Iterative Optimization. For trivial cases with very few recursive calls, optimization benefits are reduced and measurement noise becomes more significant as seen with input pair 60_20.

Table 4. Optimized GCD Performance Comparison: ARM Recursive vs x86-64 Iterative

Test Cases (a,b)	Raspberry Pi - Recursive (ns)	Linux Intel - Iterative (ns)
a = 13 b = 17	140.7	170.2
a = 60 b = 20	103.6	160
a = 55 b = 34	137.1	186.6
a = 47373888 b = 18883888	164.9	211.3

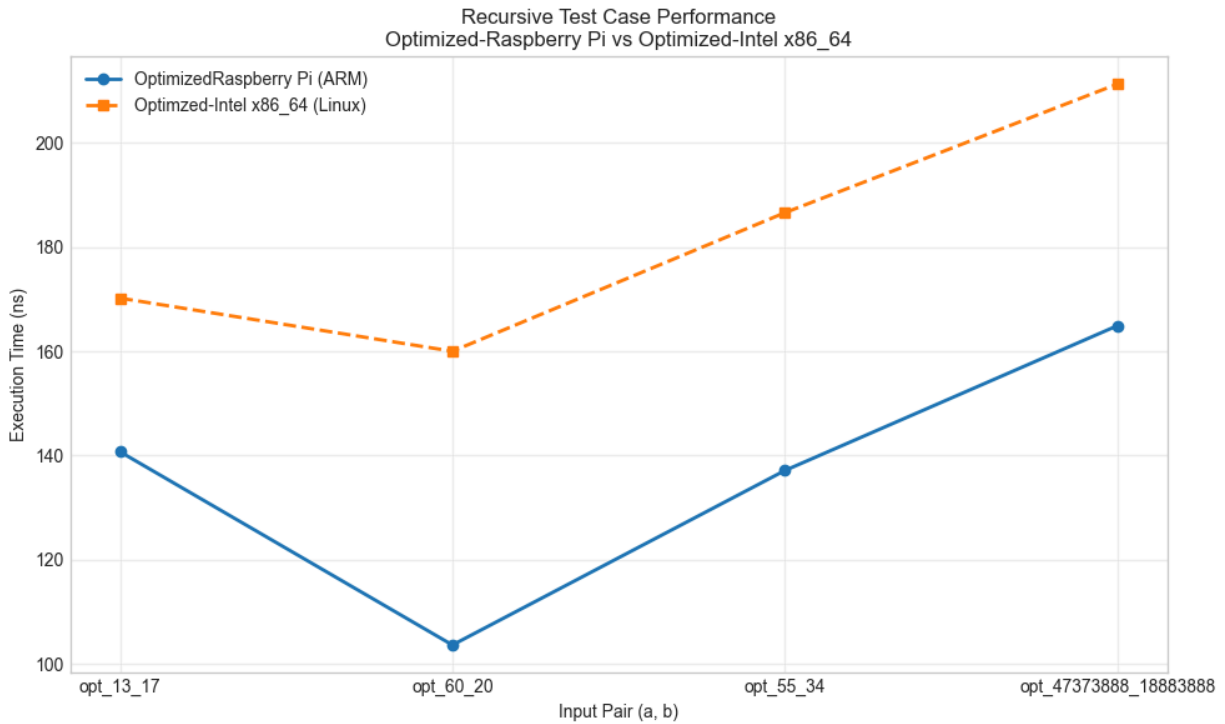


Figure 8. *Optimized GCD Performance Comparison Between ARM (Recursive) and x86-64 (Iterative)*

Summary

- Registers > Memory: Register operations are single-cycle while memory ops require multiple cycles.
- Fewer Instructions: Each instruction removed reduces CPU pipeline and fetch costs.
- Tail-Call Friendly: Prevents deep recursion stack frames, improving speed and stability.
- Better Instruction Utilization: Using instructions like the fused multiply/subtract (msub) and keeping % in edx uses the CPU's arithmetic units efficiently.

Commands used in GDB

- `gdb RecursionTest`: Opens gdb with the desired executable
- `break gcd_rekurs`: breaks the function `gcd_rekurs()`
- `run`: runs the program until the break point
- `continue`: this works in this case because it runs until the end of the next breakpoint, which in this case would be when `gcd_rekurs()` calls itself.
- `info registers sp and fp (rsp and rbp for Linux Intel)`: views the address the stack and frame pointer are currently pointing to.
- `x/200x $sp ($rsp for Linux Intel)`: does a hex dump for `$sp`. Used to examine stack memory after the recursive calls.

- frame n: n is the number of the frame. It switches to the desired frame, useful for viewing the changes of variables after each call of gcd_rekurs.
- print & a: prints the address of variable a
- print/x a: prints the value of variable a
- print &b: prints the address of variable b
- print/x b: prints the value of variable b

Appendix A - Raw Timing Data (Raspberry Pi)

- Table A.1: gcd_rekurs (Original)
- Table A.2: gcd_rekurs (Optimized)

Appendix B - Raw Timing Data (Linux Intel)

- Table B.1: gcd_rekurs (Original)
- Table B.2: gcd_rekurs (Optimized)

Table A.1: GCD_Rekurs Timing Results (Original)

Run #	a = 13 b = 17	a = 60 b = 20	a = 55 b = 34	a = 47373888 b = 18883888
Run 1	148	130	204	241
Run 2	167	112	222	241
Run 3	129	130	204	259
Run 4	166	111	185	222
Run 5	167	111	204	240
Run 6	130	111	204	222
Run 7	148	111	167	259
Run 8	166	111	166	204
Run 9	167	111	185	222
Run 10	166	111	185	223
Average	155.4	114.9	192.6	233.3

Table A.2: GCD_Rekurs Timing Results (Optimized)

Run #	a = 13 b = 17	a = 60 b = 20	a = 55 b = 34	a = 47373888 b = 18883888
Run 1	148	92	130	167
Run 2	167	111	129	166
Run 3	148	111	130	167
Run 4	167	129	130	167
Run 5	166	93	130	148
Run 6	148	112	167	167
Run 7	111	74	129	148
Run 8	111	111	130	185
Run 9	111	92	148	167
Run 10	130	111	148	167
Average	140.7	103.6	137.1	164.9

Table B.1: GCD_Recurs Timing Results (Original)

Run #	a = 13 b = 17	a = 60 b = 20	a = 55 b = 34	a = 47373888 b = 18883888
Run 1	194	162	260	294
Run 2	187	159	250	290
Run 3	185	169	254	365
Run 4	221	155	312	292
Run 5	214	132	251	293
Run 6	188	143	211	283
Run 7	184	146	250	292
Run 8	180	156	243	697
Run 9	210	156	241	289
Run 10	211	147	278	284

Average	197.4	152.5	255	337.9
----------------	--------------	--------------	------------	--------------

Table B.2: GCD_Recurs Timing Results (Optimized)

Run #	a = 13 b = 17	a = 60 b = 20	a = 55 b = 34	a = 47373888 b = 18883888
Run 1	199	187	181	216
Run 2	167	165	189	215
Run 3	163	161	182	203
Run 4	162	155	186	209
Run 5	169	153	185	205
Run 6	172	153	188	215
Run 7	169	167	182	206
Run 8	163	150	189	216
Run 9	174	155	181	222
Run 10	164	154	203	206
Average	170.2	160	186.6	211.3